Project - Low Level Design

on

Real Estate Quiz/Tutor Bot

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Samruddhi Muley	EN22CS304056
2	Payal Singh	EN22CS301701
3	Aarti Bamanke	EN22EE301001
4	Akshat Joshi	EN22CS304006
5	Punit Pawar	EN22CS303038
6	Shubhi Dubey	EN22CS304061

Group Name: 01D11

Project Number: GAI-27

Industry Mentor Name:

University Mentor Name: Dr. Hemlata Patel

Academic Year: 2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1. Introduction

The **RealEstate IQ Bot** is an AI-powered educational application designed to enhance learning in the real estate domain through interactive assessments. Unlike traditional quiz systems, the application integrates a Large Language Model (LLM) to evaluate user responses and provide contextual, domain-specific explanations.

The system focuses on **bridging knowledge gaps** by transforming each question into a learning opportunity. Instead of simply marking answers as correct or incorrect, the bot explains concepts using real-world property listing scenarios, thereby creating a **personalized and adaptive learning experience**.

The application is implemented using Python with a modular architecture, Streamlit for the user interface, and the Groq API (LLaMA 3.3 70B model) for intelligent evaluation.

1.1 Scope of the document

This document describes:

- Overall working of the RealEstate IQ Bot
- System architecture and design
- Components used in the application
- Flow of data and processes
- API usage and integration

It is useful for understanding, developing, and improving the system.

1.2 Intended Audience

This document is intended for:

- Students working on this project
- Developers who want to understand or extend the system
- Teachers or evaluators reviewing the project
- Beginners learning AI-based applications

1.3 System Overview

 # Core Functionality

- Presents users with real estate-related questions
- Accepts user responses through an interactive UI
- Evaluates answers using an LLM (LLaMA 3.3 70B via Groq API)

- Generates:
  - Score
  - Conceptual explanation
  - Real-world contextual example
  - Professional tip
- Tracks performance across topics
- Identifies weak areas for improvement

2. System Design

The RealEstate IQ Bot is designed using a **modular, layered architecture** that separates responsibilities across different components. This design ensures **scalability, maintainability, and extensibility**, allowing future enhancements such as personalization, database integration, and deployment without major structural changes.

The system integrates a **Large Language Model (LLM)** for intelligent answer evaluation, enabling contextual feedback and a personalized learning experience.

2.1 Application Design

The RealEstate IQ Bot uses a modular layered design, where each part of the system has a specific role. This makes the application easy to manage and improve.

1. Presentation Layer (UI)

Component: app.py (Streamlit)

Responsibilities:

- Displays quiz interface (Welcome, Quiz, Result, Summary)
- Captures user input (answers)
- Shows evaluation results and explanations
- Manages navigation between screens

2. Business Logic Layer

Question Bank (question_bank.py)

- Stores structured questions across topics and difficulty levels
- Provides filtered or random questions to the UI

◆ **Evaluator (evaluator.py)**

- Sends user responses to the LLM (LLaMA 3.3 via Groq API)
- Processes model output into structured JSON
- Generates:
  - Score
  - Conceptual explanation
  - Real-world example
  - Professional tip
- Implements retry logic for API reliability

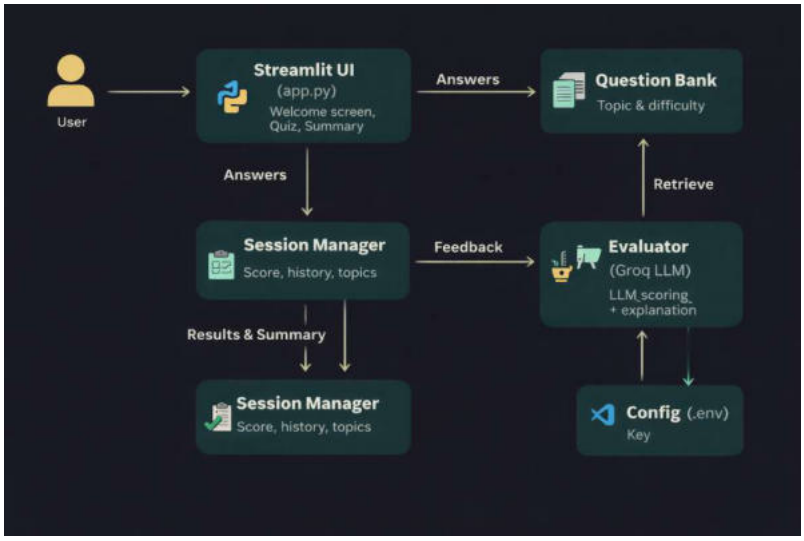
◆ **Session Manager (session_manager.py)**

- Maintains quiz state using Streamlit session state
- Tracks:
  - Total score
  - Answer history
  - Topic-wise performance
- Identifies weak topics (score < 60%)

3. Configuration Layer

Config (config.py)

- Loads environment variables from .env
- Initializes Groq API client securely



2.2 Process Flow

Step-by-step working:

The RealEstate IQ Bot follows a structured step-by-step process to provide an interactive and personalized learning experience.

Phase 1 – Core Quiz Flow (Completed)

1. **Start Application**
   The user opens the app and sees the welcome screen.
2. **Begin Quiz**
   The quiz starts with questions selected from different real estate topics and difficulty levels.
3. **Display Question**
   Each question is shown with four options (A/B/C/D), along with topic and difficulty indicators.
4. **User Input**
   The user selects an answer and clicks the submit button.
5. **AI Evaluation**
   The selected answer is sent to the AI evaluator, which checks correctness and generates:
   - Result (correct/incorrect)
   - Explanation
   - Key concept
   - Pro tip
6. **Show Result**
   The system displays feedback instantly to the user.
7. **Progress Tracking**
   The session manager updates score, history, topic-wise performance, and weak areas.
8. **Next Question**
   The process repeats until all 20 questions are completed.
9. **Summary Dashboard**
   At the end, the system shows:
   - Total score and accuracy
   - Topic-wise performance
   - Weak areas for improvement

Phase 2 – Enhanced Features (Planned)

1. **Topic & Difficulty Selection**
   Users will choose topics and difficulty level before starting the quiz, enabling personalized learning.
2. **Question Timer**
   A timer will be added to each question to simulate a real exam environment.

3.Weak Topics Re-Quiz
After the summary, users can reattempt only the topics where they performed poorly, improving their understanding.

2.3 Information Flow

- Questions are fetched from **question_bank.py**
- User input is captured in **app.py**
- Answer is sent to **evaluator.py**
- Evaluator calls Gemini API
- API returns evaluation result
- Result is sent back to UI
- Session manager updates score and progress

2.4 Components Design

1. app.py (User Interface)

- Displays questions
- Takes user input
- Shows results and dashboard

2. question_bank.py

- Contains 20 questions
- Covers 7 real estate topics

3. evaluator.py

- Sends answer to Gemini API
- Handles retries in case of API failure

4. session_manager.py

- Tracks:
  - Score
  - Current question
  - Progress
  - Weak areas

5. config.py

- Loads API key from .env file

2.5 Key Design Considerations

- User-Friendly Interface → Simple UI using Streamlit
- Modular Structure → Easy to modify and scale
- Error Handling → Handles API errors and retries
- Performance → Lightweight and fast
- Security → API key stored in .env file
- Scalability → Can add more questions or features

2.6 API Catalogue

Groq API (LLaMA 3.3 70B)

Purpose:

• Evaluate user answers using AI
• Provide contextual explanations
• Generate professional tips for learning

Key Features:

• AI-based answer evaluation
• Generates structured JSON output
• Provides real-world contextual explanations
• Supports fast inference (low latency)

API Limits (Free Tier):

• ~14,400 requests per day
• High throughput with minimal latency
• No credit card required

Error Handling:

• Automatically retries on rate limit errors
• Uses try/except for API failures
• Displays error message for non-recoverable issues

Integration Details:

• API key stored securely in .env file
• Accessed via config.py
• Used in evaluator.py for LLM interaction

3. References

- Groq
  Groq API Documentation
  Available at: https://console.groq.com/docs
- LLaMA 3
  Meta AI, LLaMA 3 Model Documentation
  Available at: https://ai.meta.com
- Prompt Engineering Guide
  Available at: https://platform.openai.com/docs/guides/prompt-engineering
- Real Estate Principles & Practices — Industry reference material